Projekt Bazy Danych 1

Aplikacja Statystyk Formuły 1

Dokumentacja Projektu

Stanisław Madziara

0. Spis Treści

1. Cel Projektu
2. Funkcjonalności
3. Akwizycja danych
4. Przepływ danych
5. Opis Struktury Bazy Danych
6. Opis Widoków Bazy Danych
7. Aplikacja Użytkownika
8. Dokumentacja Bazy Danych
9. Dokumentacja Aplikacji

1. Cel Projektu

Celem jest zbudowanie bazy danych oraz aplikacji do przeglądania, filtrowania i wyszukiwania statystyk sportu jakim jest Formuła 1.

2. Funkcjonalności

Użytkownik poprzez aplikację ma możliwość wyświetlenia dowolnej tabeli lub widoku.

Dla każdej tabeli użytkownik może wybrać które kolumny mają być wyświetlane.

Użytkownik ma możliwość posortowania wybranej tabeli po dowolnej kolumnie, oraz filtrowania wyników przez wybranie kolumny i podanie klucza wartości dla kolumny.

Użytkownik może wprowadzić, edytować lub usunąć dowolny rekord, przy założeniu że dana komenda nie narusza ograniczeń danej tabeli.

3. Akwizycja danych

Dane historyczne zostaną pobrane od przedsiębiorstwa Kaggle ze zbioru danych „Formula 1 World Championship (1950 - 2024)”.

Poniżej widnieje link do powyższego zbioru danych.

https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020

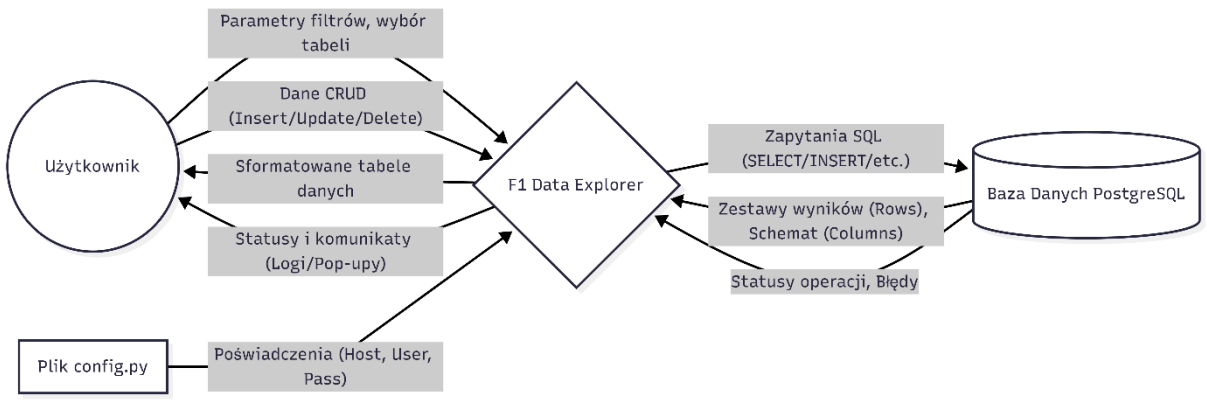
Uzyskane dane zostały przekształcone, tak aby odpowiadały strukturze bazy danych.

Następnie po zapisaniu jako pliki CSV, dane zostały zaimportowane do tabel funkcją import.

4. Przepływ danych

Cały system to dwa główne elementy – baza danych oraz aplikacja użytkownika. Użytkownik poprzez aplikację wysyła zapytanie lub polecenie CRUD o zmiennych parametrach do bazy danych. Odpowiedzią bazy danych jest komunikat/status zapytania lub dane jako odpowiedź na zapytanie. Aplikacja wyświetla otrzymane dane w głównym oknie.

Powyższa koncepcja została zilustrowana na diagramie poniżej.



Schemat 1. – DFD dla Aplikacji Statystyk Formuły 1

5. Opis Struktury Bazy Danych

Zrealizowana baza danych znajduje się w chmurze Neon Cloud. Dostęp do niej jest możliwy poprzez aplikację użytkownika.

Poniżej zdefiniowano tabele zrealizowanej bazy danych.

a) Encje Podstawowe (Słownikowe)

Tor - Reprezentuje fizyczne obiekty sportowe, na których odbywają się wyścigi.

- **ID_Toru (PK):** Unikalny identyfikator techniczny każdego toru.
- **Nazwa_Toru:** Pełna nazwa własna obiektu
- **Lokacja:** Miasto lub region, w którym znajduje się tor.
- **Kraj:** Państwo, na terenie którego położony jest obiekt.

Sezon - Określa ramy czasowe rozgrywek.

- **ID_Sezonu (PK):** Unikalny numer sezonu (rok kalendarzowy).

Zespół - Przechowuje dane o konstruktorach biorących udział w rywalizacji.

- **ID_Zespołu (PK):** Unikalny identyfikator stajni wyścigowej.
- **Nazwa_Zespołu:** Oficjalna nazwa handlowa zespołu.
- **Narodowość:** Kraj pochodzenia licencji zespołu.

Kierowca - Zawiera informacje personalne o zawodnikach.

- **ID_Kierowcy (PK):** Unikalny identyfikator zawodnika.
- **Imię / Nazwisko:** Dane osobowe kierowcy.
- **Narodowość:** Obywatelstwo zawodnika.
- **Data_Urodzenia:** Data urodzenia zawodnika.
- **Numer:** Stały numer startowy przypisany do kierowcy.

b) Encje Operacyjne (Zdarzenia)

Wyścig - Definiuje konkretne wydarzenie sportowe (Grand Prix) w ramach danego sezonu.

- **ID_Wyścigu (PK):** Unikalny identyfikator konkretnego wyścigu.
- **ID_Sezonu (FK):** Powiązanie z konkretnym sezonem.
- **ID_Toru (FK):** Wskazanie, na którym torze odbywa się ten wyścig.
- **Data_Wyścigu:** Dokładny dzień przeprowadzenia zawodów.
- **Kolejność:** Numer rundy w kalendarzu.
- **Nazwa:** Oficjalna nazwa Grand Prix.

Kierowca_Zespół - Tabela łącznikowa rozwiązująca relację wiele-do-wielu, określająca przynależność kierowcy do barw w danym czasie.

- **ID_Kierowcy (FK):** Wskazanie zawodnika.
- **ID_Zespołu (FK):** Wskazanie zespołu, w którym startuje.
- **ID_Sezonu (FK):** Określenie sezonu, w którym powyższa współpraca miała miejsce.

c) Encje Wynikowe (Statystyki)

Statystyka_Wyścigu - Dane operacyjne zbierane bezpośrednio podczas trwania wyścigu.

- **ID_Wyścigu / ID_Kierowcy (FK):** Wskazanie wyścigu / kierowcy, którego dotyczy.
- **(FK):** Wskazanie kierowcy, którego dotyczy.
- **Pozycja:** Miejsce zajęte przez kierowcę w danym okrążeniu.
- **Numer_Okrążenia:** Numer okrążenia, którego dotyczy.
- **Czas_Okrążenia:** Czas okrążenia o danym numerze.

Statystyka_Kwalifikacji - Szczegółowe wyniki sesji poprzedzających wyścig główny.

- **ID_Wyścigu / ID_Kierowcy (FK):** Wskazanie wyścigu / kierowcy.
- **Kwalifikacja_1 / 2 / 3:** Czasy uzyskane w poszczególnych etapach (Q1, Q2, Q3).
- **Pozycja_Startowa:** Miejsce na polu startowym (grid) wywalczone w kwalifikacjach.

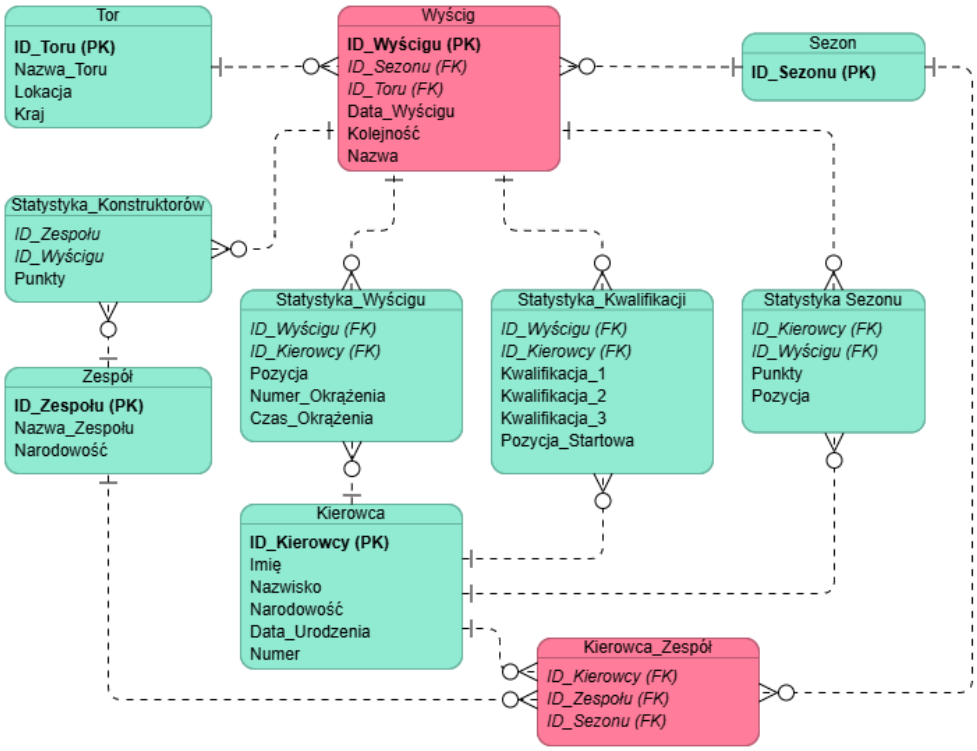
Statystyka_Sezonu - Służy do rejestrowania dorobku punktowego w ramach poszczególnych rund.

- **ID_Kierowcy / ID_Wyścigu (FK):** Wskazanie kierowcy / wyścigu.
- **Punkty:** Liczba punktów zawodnika po danym wyścigu.
- **Pozycja:** Aktualna pozycja w klasyfikacji generalnej po tym wyścigu.

Statystyka_Konstruktorów - Agreguje wyniki zespołowe (suma punktów obu kierowców zespołu).

- **ID_Zespołu / ID_Wyścigu (FK):** Wskazanie zespołu i konkretnego wyścigu.
- **Punkty:** Łączna zdobycz punktowa zespołu w danej rundzie.

Poniżej przedstawiono schemat ERD bazy danych.



Schemat 2. – ERD Bazy Danych Statyskyk Formuły 1

6. Opis Widoków Bazy Danych

Poniżej zdefiniowano widoki bazy danych wraz z opisem funkcjonalnym.

Komendy wykorzystane do utworzenia tabel znajdują się w pliku *Widoki_Dokumentacja.pdf*.

a) Widok_Ilość_Zwycięstw

Widok ten służy do rankingowego zestawienia kierowców pod kątem ich sukcesów w wyścigach.

Łączy tabelę Statystyka_Sezonu z tabelą Kierowca. Wykorzystuje funkcję agregującą count z filtrowaniem (FILTER (WHERE "Pozycja" = 1)) oraz funkcję concat do złączenia imienia i nazwiska w jedno pole tekstowe. Wyniki są prezentowane malejąco według liczby zwycięstw.

b) Widok_Kierowca_Zespół

Podstawowy widok informacyjny mapujący przynależność zawodników do konkretnych stajni wyścigowych.

Integruje trzy tabele: Kierowca_Zespół, Kierowca oraz Zespół. Pozwala na szybkie zidentyfikowanie, w jakich barwach startował dany kierowca w wybranym roku. Dane są ułożone chronologicznie według identyfikatora sezonu oraz alfabetycznie według nazw zespołów.

c) Widok_Podsumowanie_Konstruktorów

Zaawansowany widok analityczny przedstawiający kondycję punktową zespołów.

Agreguje punkty z tabeli Statystyka_Konstruktorów, łącząc je z tabelami Zespół i Wyścig. Sumuje punkty dla każdego zespołu w ramach danego sezonu. Grupowanie odbywa się według sezonów, a wewnątrz nich zespoły uszeregowane są od tego z największą liczbą punktów.

d) Widok_Statystyka_Konstruktorów

Widok szczegółowy dotyczący występów zespołów w konkretnych wyścigach.

Łączy dane o punktach (Statystyka_Konstruktorów) z nazwą wyścigu, jego kolejnością w kalendarzu oraz nazwą zespołu. Zapewnia chronologiczny podgląd postępów sezonu (ID_Sezonu, Kolejność).

e) Widok_Zwycięzcy

Reprezentacyjny widok historyczny podsumowujący zwycięzców poszczególnych rund.

Wykorzystuje złączenie czterech tabel: Wyścig, Tor, Statystyka_Sezonu oraz Kierowca. Kluczowym elementem jest klauzula WHERE, która filtruje dane wyłącznie dla zawodników zajmujących 1. pozycję. Prezentuje dane w porządku chronologicznym (sezon po sezonie, wyścig po wyścigu).

7. Aplikacja Użytkownika

Aplikacja została wykonana w języku Python, z wykorzystaniem bibliotek customtkinter i psycopg2.

Aby uruchomić aplikację należy w wywołać plik *Aplikacja_F1.py*.

Pliki aplikacji do poprawnego działania muszą mieć możliwość połączenia z bazą danych znajdującą się w chmurze.

Interfejs główny składa się z 4 obszarów – Panel Selekcji, Panel Filtrowania, Panel CRUD oraz Ekran Wynikowy.

a) Panel Selekcji

Znajduje się po lewej stronie ekranu, zawiera przycisk do przejścia do ekranu CRUD, pole wyboru tabeli / widoku, pole wyboru kolumn w tabeli oraz dwa przyciski do inkrementacji / dekrementacji wyświetlanej strony.

b) Panel Filtrowania

Znajduje się na dole ekranu, zawiera pole wyboru kolumny sortowania, pole wyboru porządku sortowania, pole wyboru kolumny filtrowania, pole wprowadzenia klucza filtrowania oraz przycisk do zaaplikowania wprowadzonych zmian w zapytaniu.

c) Panel CRUD

Znajduje się na szczycie Panelu Selekcji, po jego kliknięciu otwiera się okienko w którym można wybrać tabelę do edycji, wprowadzić wartości dla kolumn, wybrać kolumnę filtra oraz wartość filtra. Na dole okienka znajdują się trzy przyciski które umożliwiają dodanie, edytowanie lub usunięcie rekordu o wcześniej wprowadzonych parametrach. Użytkownik jest powiadamiany o sukcesie lub niepowodzeniu akcji komunikatem na dole okienka.

d) Ekran Wyświetlania

Zajmuje centralną część okna, wyświetla wynik zapytania wysłanego do bazy danych. Jest automatycznie aktualizowany po zmianie wyświetlanej tabeli. W przypadku wykonania innej akcji należy użyć przycisku w Panelu Filtrowania do aktualizacji wyświetlanych danych.

8. Dokumentacja Bazy Danych

Poniżej znajdują się polecenia SQL do utworzenia tabel i widoków składających się na bazę danych.

1) Kierowca

```
CREATE TABLE IF NOT EXISTS public."Kierowca" (
  "ID_Kierowcy" integer NOT NULL DEFAULT nextval('kierowca_id_kierowcy_seq'::regclass),
  "Imię" text COLLATE pg_catalog."default" NOT NULL,
  "Nazwisko" text COLLATE pg_catalog."default" NOT NULL,
  "Narodowość" text COLLATE pg_catalog."default" NOT NULL,
  "Data_Urodzenia" date NOT NULL,
  "Numer" integer,
  CONSTRAINT kierowca_pkey PRIMARY KEY ("ID_Kierowcy"),
  CONSTRAINT unikalny_kierowca UNIQUE ("Imię", "Nazwisko"))
```

2) Kierowca_Zespół

```
CREATE TABLE IF NOT EXISTS public."Kierowca_Zespół" (
  "ID_Kierowcy" integer,
  "ID_Zespołu" integer,
  "ID_Sezonu" integer)
```

3) Sezon

```
CREATE TABLE IF NOT EXISTS public."Sezon" (
  "ID_Sezonu" integer NOT NULL,
  CONSTRAINT sezon_pkey PRIMARY KEY ("ID_Sezonu"))
```

4) **Statystyka_Konstruktorów**

```
CREATE TABLE IF NOT EXISTS public."Statystyka_Konstruktorów" (
  "ID_Zespołu" integer NOT NULL,
  "ID_Wyścigu" integer NOT NULL,
  "Punkty" numeric(5,2))
```

5) **Statystyka_Kwalifikacji**

```
CREATE TABLE IF NOT EXISTS public."Statystyka_Kwalifikacji" (
  "ID_Wyścigu" integer NOT NULL,
  "ID_Kierowcy" integer NOT NULL,
  "Kwalifikacja_1" interval,
  "Kwalifikacja_2" interval,
  "Kwalifikacja_3" interval,
  "Pozycja_Startowa" integer )
```

6) **Statystyka_Sezonu**

```
CREATE TABLE IF NOT EXISTS public."Statystyka_Sezonu" (
  "ID_Kierowcy" integer NOT NULL,
  "ID_Wyścigu" integer NOT NULL,
  "Punkty" numeric(5,2) NOT NULL,
  "Pozycja" integer NOT NULL)
```

7) **Statystyka_Wyścigu**

```
CREATE TABLE IF NOT EXISTS public."Statystyka_Wyścigu" (
  "ID_Wyścigu" integer NOT NULL,
  "ID_Kierowcy" integer NOT NULL,
  "Pozycja" integer,
  "Numer_Okrążenia" integer NOT NULL,
  "Czas_Okrążenia" interval)
```

8) **Tor**

```
CREATE TABLE IF NOT EXISTS public."Tor" (
  "ID_Toru" integer NOT NULL DEFAULT nextval('tor_id_toru_seq'::regclass),
  "Nazwa_Toru" text COLLATE pg_catalog."default" NOT NULL,
  "Lokacja" text COLLATE pg_catalog."default" NOT NULL,
  "Kraj" text COLLATE pg_catalog."default" NOT NULL,
  CONSTRAINT tor_pkey PRIMARY KEY ("ID_Toru"),
  CONSTRAINT tor_nazwa_toru_key UNIQUE ("Nazwa_Toru"))
```

9) **Wyścig**

```
CREATE TABLE IF NOT EXISTS public."Wyścig" (
  "ID_Wyścigu" integer NOT NULL DEFAULT nextval('wyscig_id_wyscigu_seq'::regclass),
  "ID_Sezonu" integer NOT NULL,
  "ID_Toru" integer NOT NULL,
  "Data_Wyścigu" date NOT NULL,
  "Kolejność" integer,
  "Nazwa" text COLLATE pg_catalog."default",
  CONSTRAINT wyscig_pkey PRIMARY KEY ("ID_Wyścigu"))
```

10) Zespół

```
CREATE TABLE IF NOT EXISTS public."Zespół" (
  "ID_Zespołu" integer NOT NULL DEFAULT nextval('"zespół_id_zespołu_seq"'::regclass),
  "Nazwa_Zespołu" text COLLATE pg_catalog."default" NOT NULL,
  "Narodowość" text COLLATE pg_catalog."default" NOT NULL,
  CONSTRAINT "zespół_pkey" PRIMARY KEY ("ID_Zespołu"),
  CONSTRAINT "zespół_nazwa_zespołu_key" UNIQUE ("Nazwa_Zespołu"))
```

11) Widok_Ilość_Zwycięstw

```
CREATE OR REPLACE VIEW public."Widok_Ilość_Zwycięstw"
 AS
 SELECT concat(k."Imię", ' ', k."Nazwisko") AS kierowca,
   count(ss."Pozycja") FILTER (WHERE ss."Pozycja" = 1) AS "Zwycięstwa"
  FROM "Statystyka_Sezonu" ss
    JOIN "Kierowca" k ON ss."ID_Kierowcy" = k."ID_Kierowcy"
 GROUP BY (concat(k."Imię", ' ', k."Nazwisko"))
 ORDER BY (count(ss."Pozycja") FILTER (WHERE ss."Pozycja" = 1)) DESC;
```

12) Widok_Kierowca_Zespół

```
CREATE OR REPLACE VIEW public."Widok_Kierowca_Zespół"
 AS
 SELECT kz."ID_Sezonu",
   z."Nazwa_Zespołu",
   concat(k."Imię", ' ', k."Nazwisko") AS kierowca
  FROM "Kierowca_Zespół" kz
    JOIN "Kierowca" k ON k."ID_Kierowcy" = kz."ID_Kierowcy"
    JOIN "Zespół" z ON z."ID_Zespołu" = kz."ID_Zespołu"
 ORDER BY kz."ID_Sezonu", z."Nazwa_Zespołu";
```

13) Widok_Podsumowanie_Konstruktorów

```
CREATE OR REPLACE VIEW public."Widok_Podsumowanie_Konstruktorów"
 AS
 SELECT w."ID_Sezonu",
   z."Nazwa_Zespołu",
   sum(sk."Punkty") AS "Suma_Punktów"
  FROM "Statystyka_Konstruktorów" sk
    JOIN "Zespół" z ON sk."ID_Zespołu" = z."ID_Zespołu"
    JOIN "Wyścig" w ON w."ID_Wyścigu" = sk."ID_Wyścigu"
 GROUP BY w."ID_Sezonu", z."Nazwa_Zespołu"
 ORDER BY w."ID_Sezonu", (sum(sk."Punkty")) DESC;
```

14) Widok_Statystyka_Konstruktorów

```
CREATE OR REPLACE VIEW public."Widok_Statystyka_Konstruktorów"
 AS
 SELECT w."ID_Sezonu",
   w."Kolejność",
   w."Nazwa",
   z."Nazwa_Zespołu",
   sk."Punkty"
  FROM "Statystyka_Konstruktorów" sk
    JOIN "Zespół" z ON sk."ID_Zespołu" = z."ID_Zespołu"
    JOIN "Wyścig" w ON w."ID_Wyścigu" = sk."ID_Wyścigu"
 ORDER BY w."ID_Sezonu", w."Kolejność", z."Nazwa_Zespołu";
```

15) Widok_Zwycięzcy

```
CREATE OR REPLACE VIEW public."Widok_Zwycięzcy"
 AS
 SELECT w."ID_Sezonu",
   w."Kolejność",
   w."Data_Wyścigu",
   t."Nazwa_Toru",
   w."Nazwa",
   concat(k."Imię", ' ', k."Nazwisko") AS "Zwycięzca"
  FROM "Wyścig" w
    JOIN "Tor" t ON w."ID_Toru" = t."ID_Toru"
    JOIN "Statystyka_Sezonu" ss ON w."ID_Wyścigu" = ss."ID_Wyścigu"
    JOIN "Kierowca" k ON k."ID_Kierowcy" = ss."ID_Kierowcy"
 WHERE ss."Pozycja" = 1
 ORDER BY w."ID_Sezonu", w."Kolejność";
```

9. Dokumentacja Aplikacji

Poniżej znajduje się opis klas oraz ich metod składających się na aplikację użytkownika.

1) Moduł MenadzerBazyDanych.py

Główna klasa odpowiedzialna za niskopoziomową komunikację z serwerem PostgreSQL.

- __init__(self): Inicjalizuje atrybut self.polaczenie wartością None, przygotowując obiekt do nawiązania sesji.
- polacz(self): Sprawdza, czy połączenie istnieje lub czy nie zostało zamknięte; jeśli wymagane, ustanawia nowe połączenie, korzystając z parametrów z pliku config. Zwraca True przy sukcesie lub False w przypadku błędu psycopg2.Error.
- _wykonaj(self, zapytanie, parametry=None, pobierz=False): Prywatna metoda generyczna zarządzająca cyklem życia kursora. Obsługuje zatwierdzanie transakcji (commit), wycofywanie zmian przy błędzie (rollback) oraz zwraca nagłówki i wiersze, jeśli parametr pobierz jest ustawiony na True.

- pobierz_tabele(self): Wykonuje zapytanie do information_schema.tables w celu pobrania nazw wszystkich tabel bazowych i widoków w schemacie publicznym.
- pobierz_kolumny(self, nazwa_tabeli): Pobiera listę nazw kolumn dla konkretnej tabeli z information_schema.columns. Wykorzystuje sql.Literal dla bezpiecznego przekazania nazwy tabeli.
- pobierz_dane_tabeli(self, nazwa_tabeli, kolumny=None, sortuj_po=None, rosnaco=True, limit=50, przesuniecie=0, kolumna_filtra=None, wartosc_filtra=None): Buduje dynamiczne zapytanie SELECT. Obsługuje wybór konkretnych pól, filtrowanie przez rzutowanie na TEXT, sortowanie oraz pagynację przy użyciu klauzul LIMIT i OFFSET.
- pobierz_sume_stron(self, nazwa_tabeli, rozmiar_strony, kolumna_filtra=None, wartosc_filtra=None): Wykonuje COUNT(*) z uwzględnieniem filtrów, a następnie oblicza całkowitą liczbę stron przy użyciu funkcji math.ceil.
- wykonaj_wstawianie(self, nazwa_tabeli, slownik_danych): Generuje zapytanie INSERT INTO. Mapuje klucze słownika na nazwy kolumn i przygotowuje odpowiednią liczbę symboli zastępczych (%s) dla wartości.
- wykonaj_usuwanie(self, nazwa_tabeli, kolumna_warunku, wartosc_warunku): Realizuje operację DELETE FROM. Filtr warunku jest rzutowany na tekst, co pozwala na usuwanie rekordów na podstawie identyfikatorów podanych w GUI jako string.
- wykonaj_aktualizacje(self, nazwa_tabeli, slownik_danych, kolumna_warunku, wartosc_warunku): Filtruje słownik w celu usunięcia pustych wartości, po czym buduje zapytanie UPDATE z klauzulą SET dla zmienionych pól.

2) Moduł AplikacjaF1.py

Klasa AplikacjaF1

Główne okno aplikacji zarządzające interfejsem użytkownika i wyświetlaniem danych.

- __init__(self): Konfiguruje parametry okna, inicjalizuje obiekt MenadzerBazyDanych oraz wywołuje budowę interfejsu i wstępne ładowanie kolumn.
- _konfiguruj_interfejs(self): Definiuje układ siatki (grid), tworzy panel boczny, ramkę nawigacji, sekcję filtrów oraz przewijalny obszar wyników.
- aktualizuj_liste_kolumn(self, nazwa_tabeli): Czyści poprzednie kontrolki, pobiera nowe kolumny z bazy i tworzy dla nich checkboxy. Aktualizuje również opcje w menu sortowania i filtrowania.

- zmien_strone(self, delta): Modyfikuje indeks aktualnej strony o przekazaną wartość delta i odświeża widok wyników.
- zastosuj_filtry(self): Resetuje licznik stron do zera i wywołuje odświeżenie danych na podstawie aktualnie ustawionych filtrów.
- wyswietl_wyniki(self): Główna funkcja renderująca. Pobiera stan wszystkich kontrolek (wybrane kolumny, filtry, sortowanie), pobiera dane z bazy i dynamicznie buduje tabelę z etykiet CTkLabel w ramce wyników.
- otworz_crud(self): Inicjalizuje i wyświetla nowe okno klasy OknoCRUD.

3) Moduł crud.py

Klasa OknoCRUD

Okno dialogowe do bezpośredniego manipulowania rekordami w bazie.

- __init__(self, rodzic, menadzer_bazy): Ustawia okno jako topmost, inicjalizuje interfejs i generuje pola wejściowe dla domyślnie wybranej tabeli.
- _konfiguruj_interfejs(self): Tworzy menu wyboru tabeli, obszar formularza, pola warunku WHERE oraz przyciski akcji (Insert, Update, Delete).
- generuj_pola(self, nazwa_tabeli): Dynamicznie tworzy pary Label i Entry dla każdej kolumny w wybranej tabeli, przechowując referencje do pól tekstowych w słowniku self.pola_wprowadzania.
- pokaz_status(self, wiadomosc, kolor="white"): Aktualizuje tekst i kolor etykiety statusu na dole okna w celu poinformowania użytkownika o wyniku operacji.
- akcja_wstawiania(self): Pobiera dane z formularza, konwertując puste pola na None, i przekazuje je do metody wykonaj_wstawianie w menedżerze bazy.
- akcja_usuwania(self): Wyświetla okno potwierdzenia askyesno; po akceptacji wywołuje usuwanie rekordu na podstawie wartości z pola warunku.
- akcja_aktualizacji(self): Zbiera dane z pól tekstowych i wykonuje aktualizację rekordu spełniającego kryterium z pola warunku WHERE.